

University of Constantine 2 University - Abdelhamid Mehri
Department of Fundamental Computing and its Applications - IFA
License in Computer Science - 3rd Year
Course: IAM
2025-2026

TP 04: Flutter UI Fundamentals & Modern Component Design

General Requirements

Custom Branding: For every exercise, generate a unique app icon using appicon.co to ensure the app feels "production-ready" on the emulator.

Design Standards: Use modern design principles (Material 3) and professional typography (**Nunito** or **DM Sans**).

Exercise 01: The Reactive "Smart Home" Dashboard

Objective: Master local state management using **setState** and **ValueNotifier**.

- **Task:** Create a single-page Smart Home dashboard.
- **Features:**
 - **Device Toggles:** Create custom widgets for "Light," "AC," and "Wi-Fi." Use **setState** to toggle their active/inactive visual states.
 - **Live Slider:** Use a **ValueNotifier<double>** to control a "Brightness" or "Temperature" slider. The UI should update reactively without rebuilding the entire scaffold.
 - **Dynamic Stats:** Show a "Power Usage" card that changes color (green to red) based on how many devices are toggled on.

Exercise 02: Multi-Screen Navigation & Global Profile

Objective: Implement **Provider** for basic dependency injection and cross-screen data sharing.

- **Task:** Build a multi-page app with a centralized "User Preferences" state.
- **Features:**
 - **The AppBar Actions:** Include three distinct icons: **Profile**, **Settings**, and **Notifications**.
 - **Global Theme State:** Use **ChangeNotifierProvider** to allow the user to switch between Light, Dark, and "High Contrast" modes across all three screens.
 - **Profile Persistence:** Data entered in the "Settings" page (Username, Bio) must instantly reflect on the "Profile" page without passing constructor arguments.

Exercise 03: The "Crypto-Pulse" Tracker (BLoC Basics)

Objective: Transition to BLoC for handling asynchronous events and complex UI states.

Task: Build a real-time cryptocurrency price tracker (using mocked data).

Features:

- **State-Driven UI:** Define three BLoC states: **CryptoLoading**, **CryptoLoaded**, and **CryptoError**.
- **Interaction:** A "Refresh" FloatingActionButton that triggers a **FetchPrices** event.
- **Conditional Rendering:** Use **BlocBuilder** to show a Shimmer effect during loading and a **ListView** of coins with prices when loaded.
- **The "Dice" Logic:** Add a "Randomize Portfolio" button that randomly generates a new set of coin values, simulating a dice-roll logic for price fluctuations.

Exercise 04: The Attendance Pro (Local Persistence & BLoC)

Objective: Manage complex lists and local "database-like" state using BLoC.

- **Task:** A teacher's tool to track student attendance across multiple sessions.
- **Features:**
 - **Interactive Student Cards:** Each card features a profile image and 5 "Session Chips." Tapping a chip toggles its color (Present/Absent) via a **ToggleAttendance** event.
 - **Dynamic Adding:** Use a **BottomSheet** or a new screen with a Form to add a new student to the BLoC's list dynamically.
 - **Statistics Header:** A top bar that calculates the "Class Attendance Average" in real-time as icons are checked/unchecked.

Exercise 05: The Secure Vault (Auth & Data Flow)

Objective: Implement a full Login-to-Profile flow using BLoC and the Repository Pattern.

- **Task:** Create a secure gateway app that validates credentials against a simulated or local database.
- **Features:**
 - **Validation Logic:** Use `TextEditingController` to capture input. The BLoC should emit a `ValidationError` if the password is less than 8 characters or if the email format is incorrect.
 - **Secure Transition:** Upon successful login, show a Success Dialog, then navigate to a "Dashboard" using `BlocListener`.
 - **Data Injection:** The Dashboard must fetch the user's detailed "Profile Data" (ID, Role, Join Date) from a Repository based on the ID saved in the Auth state.